



MERRIMACK COLLEGE

CSC 6033

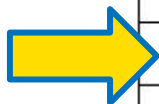
Week 3

Finite Automata

Languages, Automata, and Decidability - Dr. Paulo Fernandes

Timeline

Unit	Week	Topic	Projects	Quizzes	In-class Exercises	Exam
1	1	From Python to C++	P#1	Q#1	E#1	
1	2	Developing with C++	P#2	Q#2	E#2	
2	3	Finite Automata	P#3	Q#3	E#3	
2	4	Regular Expressions	P#4	Q#4	E#4	
3	5	Pushdown Automata	P#5	Q#5	E#5	
4	6	Turing Machines	P#6	Q#6	E#6	
5	7	Processing Input and Output	P#7	Q#7	E#7	
6	8	Decidability				Final Exam (all units)



Where are we?



MERRIMACK COLLEGE

Don't let tasks pile up!

Presentation Agenda

Week 3

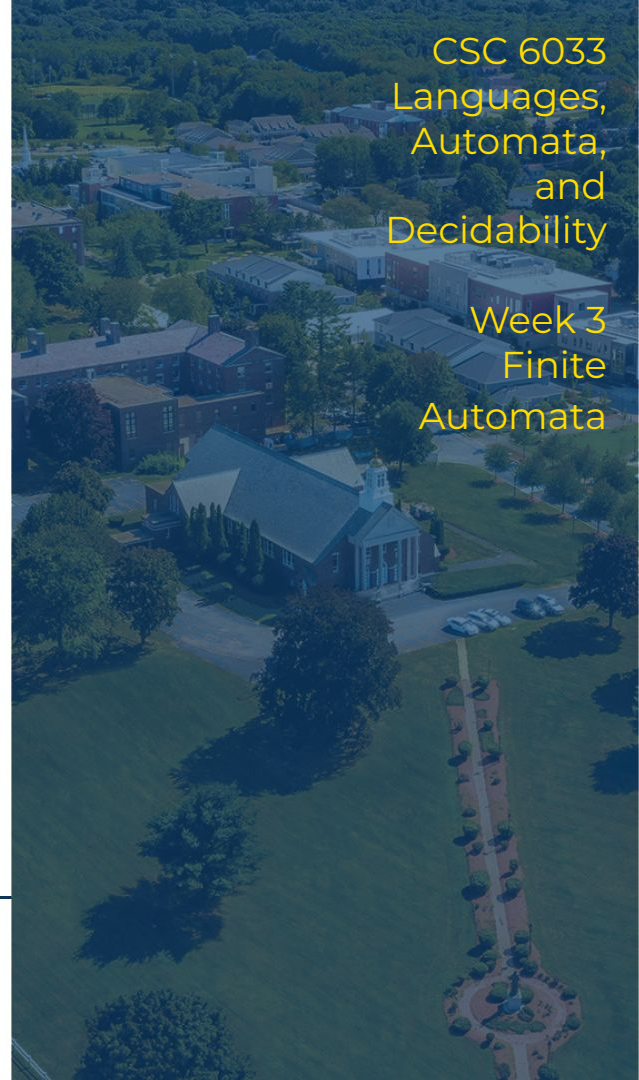
- Models of Computation
 - a. Theory of Computation
 - b. Languages
 - c. Examples
- Finite Automata
 - a. Deterministic and Non-Deterministic
 - b. Models of Computation Hierarchy
 - c. Implementation
- This Week's tasks



MERRIMACK COLLEGE

CSC 6033
Languages,
Automata,
and
Decidability

Week 3
Finite
Automata



Models of Computation

n Mathematicians are a bit like the laconic Vermonter who, when asked if he's lived in the state his whole life, replies, 'Not yet.'

John Allen Paulos

Since you are here in this course, you have been working with programming and discrete mathematics for a while.

However, it is only now that you will be aware of what computation is about, *i.e.*, what indeed computers can do, from a mathematical and logical point of view.

- **Theory of Computation**
- Languages
- Examples



Theory of Computation

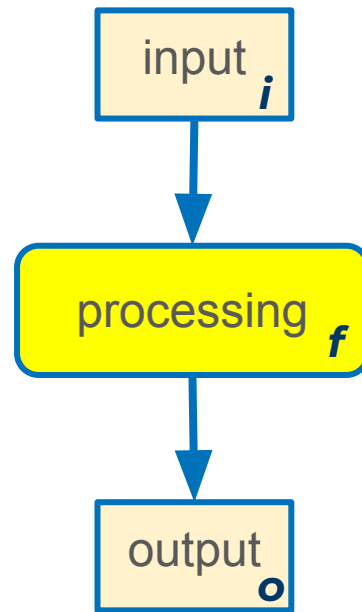
A basic definition of computers is:

A machine that receives some input, performs some processing and delivers some output.

Transforming that definition into a formal mathematical analysis, we can consider a computer as a function **$f()$** receiving an input **i** and delivering an output **o** :

$$f(i) = o \quad \text{or} \quad f: i \mapsto o$$

As such, a *model of computation* is a model which describes how an output of a mathematical function is computed given an input.

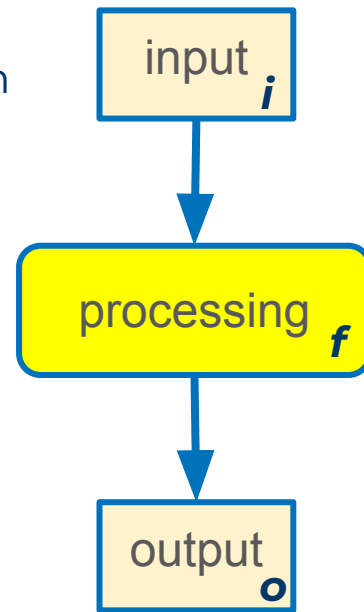
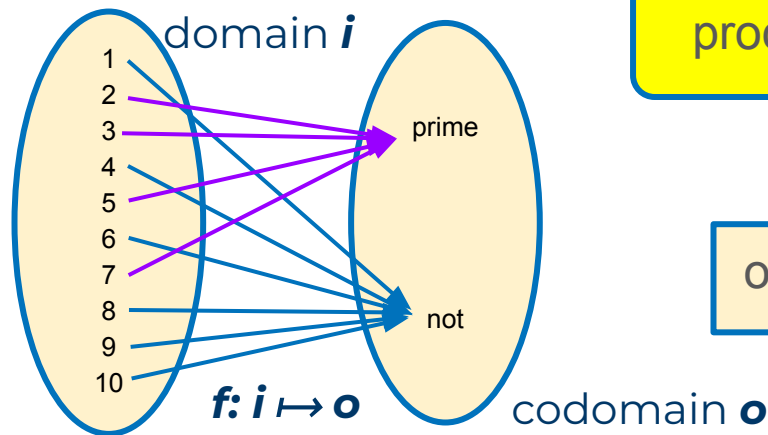


A Simple Model of Computation

Perhaps the simplest way to represent a model of computation is to state what to do given an input. For example, let's create a model of computation that receives as input a number from 1 to 10 and outputs if it is a prime number or not.

input i	output o
1	not
2	prime
3	prime
4	not
5	prime

input i	output o
6	not
7	prime
8	not
9	not
10	not



A Less Simple Model of Computation

The direct function mode is ok when you have a reasonably small number of inputs, but what if the input possibilities is larger?



For example, assuming that you have only dimes (10 cents) and nickels (5 cents), let's imagine a function that receives a certain number of coins in a specific order and decide if they add up 25 cents with the initial coins.

The theoretical number of inputs is infinite, even though we can enumerate a certain number of valid inputs:



- 5, 5, 5, 5, 5, ... are valid inputs;
- 5, 10, 10, ... are valid inputs;
- 10, 5, 10, ... are valid inputs;
- 10, 10, 5, ... are valid inputs;
- 5, 5, 5, 10, ... are valid inputs;
- 5, 5, 10, 5, ... are valid inputs;
- 5, 10, 5, 5, ... are valid inputs;
- 10, 5, 5, 5, ... are valid inputs.



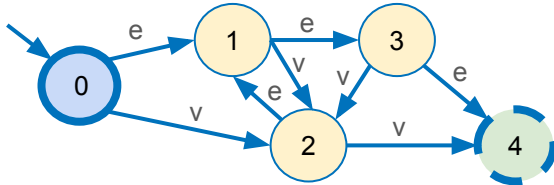
(Automata)

Automata can be all sort of automated devices that change their state according to rules, as for example:

- ATM - automated teller machine;
- Swiss Clock (elaborated clockwork);
- Automaton (mechanical movement).



We can represent it using Directed Graphs.



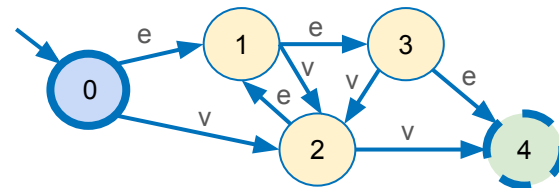
MERRIMACK COLLEGE

Automaton (sing.) - Automata (plural)

(Finite Automata)

A finite automata is formally defined as a 5-tuple:

- **Q** - a finite set of states
 - **$\{0, 1, 2, 3, 4\}$**
- **Σ** - a finite set of symbols (a.k.a. alphabet)
 - **$\{e, v\}$**
- **δ** - a transition function defining edges between states according to symbols
 - **$\{0 \rightarrow 1 (e), 0 \rightarrow 2 (v), 1 \rightarrow 2 (v), 1 \rightarrow 3 (e), 2 \rightarrow 1 (e), 2 \rightarrow 4 (v), 3 \rightarrow 2 (v), \text{ and } 3 \rightarrow 4 (e)\}$**
- **$q_0 \in Q$** - one initial state
 - **0**
- **$F \subseteq Q$** - a set of final states
 - **$\{4\}$**



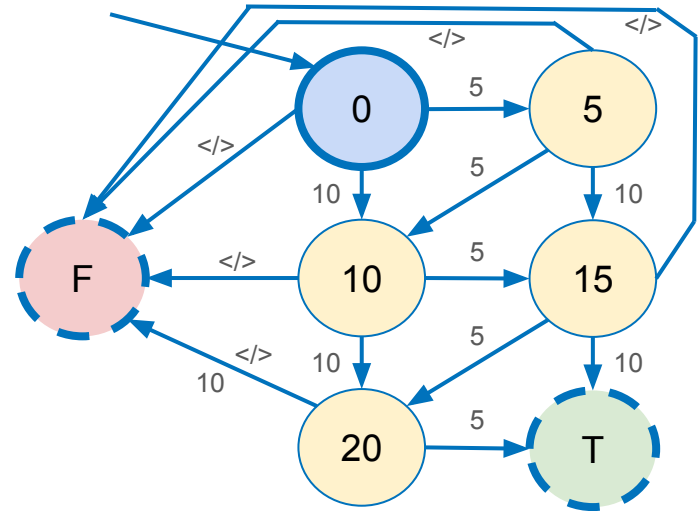
A Model of Computation

Considering only dimes and nickels, what would be an algorithm to decide if a certain number of coins in a specific order adds up to 25 cents with the initial coins.

- 5, 5, 5, 5, 5, ... are valid inputs;
- 5, 10, 10, ... are valid inputs;
- 10, 5, 10, ... are valid inputs;
- 10, 10, 5, ... are valid inputs;
- 5, 5, 5, 10, ... are valid inputs;
- 5, 5, 10, 5, ... are valid inputs;
- 5, 10, 5, 5, ... are valid inputs;
- 10, 5, 5, 5, ... are valid inputs.



</> no more coins



FSA (or FSM)

Finite State Automaton
a.k.a.
Finite State Machine



MERRIMACK COLLEGE

Video: [Computers without memory.](#)

Models of Computation

n Mathematicians are a bit like the laconic Vermonter who, when asked if he's lived in the state his whole life, replies, 'Not yet.'"

John Allen Paulos

Since you are here in this course, you have been working with programming and discrete mathematics for a while.

However, it is only now that you will be aware of what computation is about, *i.e.*, what indeed computers can do, from a mathematical and logical point of view.

- Theory of Computation
- **Languages**
- Examples



Input as a Language

The kind of input submitted to a FSA can be seen as words of a language.

A language is composed by an alphabet Σ , the basic elements composing the words. For the example:

- Dime (**10**)
- Nickel (**5**), and
- The end of coins (**</>**).

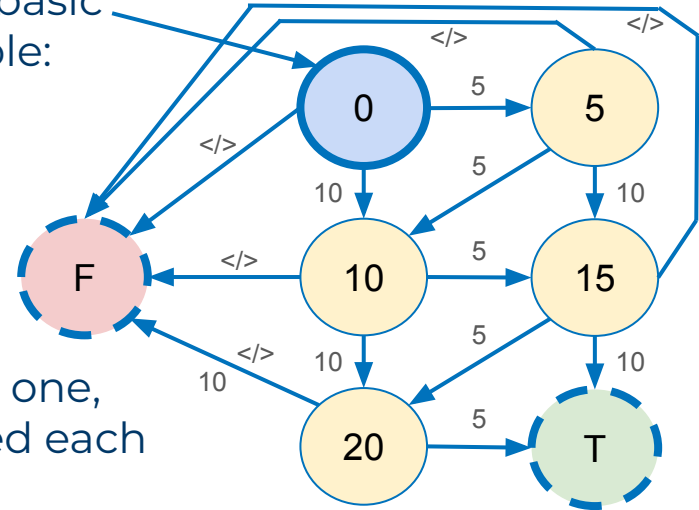
$$\Sigma = \{ 5, 10 \}$$

The valid words would be defined by the FSA.

In this example, the 7 states, defining the initial one, the final ones, and the transitions to be executed each time an alphabet element is received.



</> no more coins



Formal Languages

Formally, a FSA is capable to define a language as the inputs that it can recognize (detect, acknowledge, deliver a True output).

A formal language L is defined over an alphabet Σ , by either:

- a set of acceptable words:
 - like a lexikon (dictionary without the meaning of words);
- a set of rules to generate the acceptable words:
 - like a grammar;
- a set of rules to decide if a word belongs to the acceptable ones:
 - like a FSA.

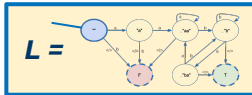
$$\Sigma = \{a, b\}$$

Words over the Alphabet

a, b, aa, ab, ba, bb, aaa, aab, aba, abb,
baa, bab, bba, bbb, aaaa, aaab, ...

$$L = \{aab, aaab, aaba, aabb, aaaab, \dots\}$$

L = words starting with "aa",
but not ending with "aa".



Language Definition

A formal language L is defined over an alphabet Σ , by either:

$\Sigma = \{ a, b \}$

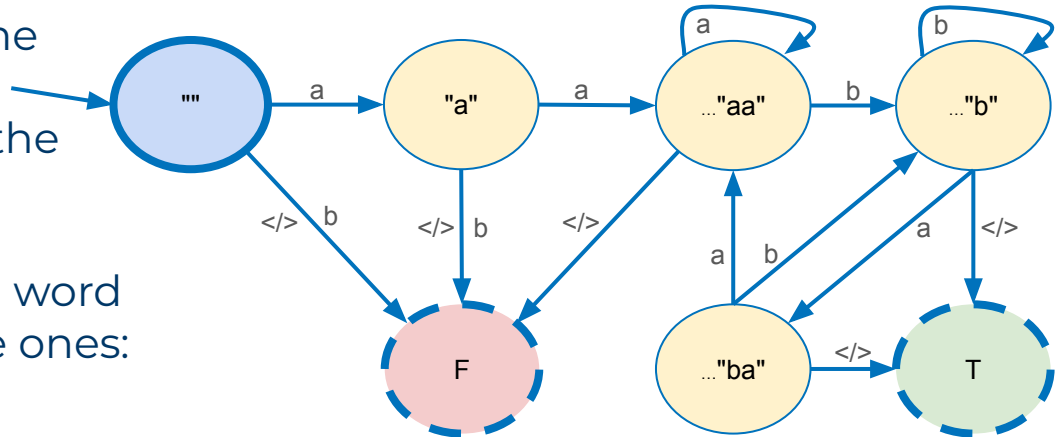
Words over the Alphabet

a, b, aa, ab, ba, bb, aaa, aab, aba, abb, baa, bab, bba, bbb, aaaa, aaab, ...

- a set of acceptable words:
 - like a lexikon (dictionary without the meaning of words);
- a set of rules to generate the acceptable words:
 - like a grammar;
- a set of rules to decide if a word belongs to the acceptable ones:
 - like a FSA.

$L = \{ aab, aaab, aaba, aabb, aaaab, \dots \}$

$L =$ words starting with "aa", but not ending with "aa".



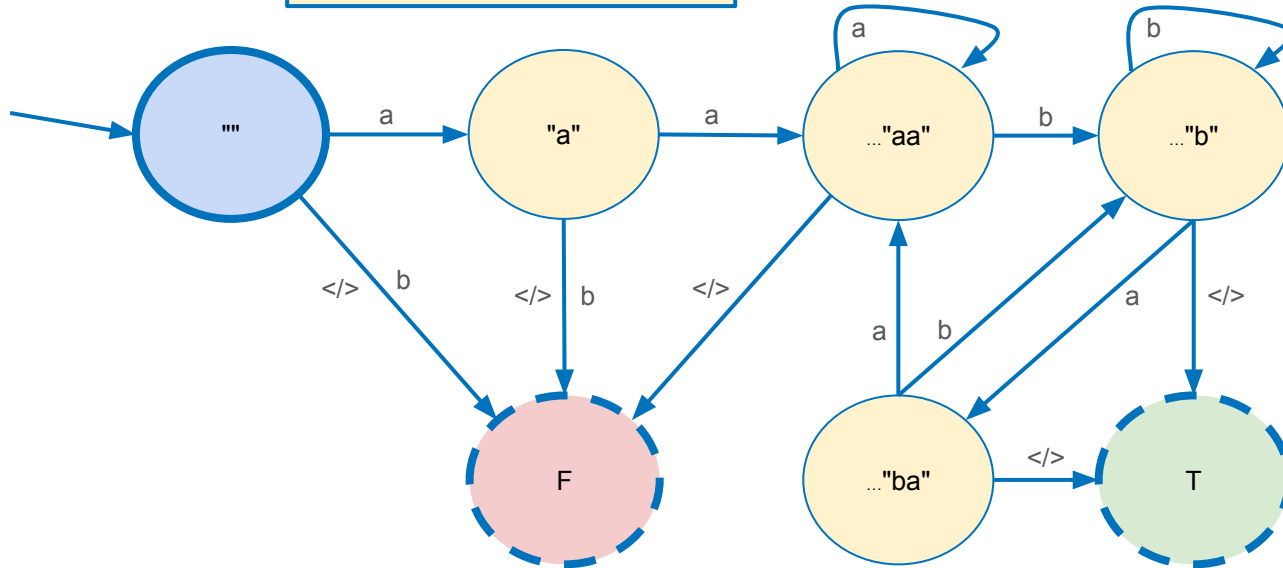
MERRIMACK COLLEGE

A formal language, in this context, is a language that we can infer all words belonging to it.

Focus on this FSA

$\Sigma = \{a, b\}$

L = words starting with "aa", but not ending with "aa".



- ""
- "a"
- "b"
- "aa"
- "ab"
- "ba"
- "bb"
- "aaa"
- "aab"
- "aba"
- "abb"
- "baa"
- "bab"
- "bba"
- "bbb"
- "aaaa"
- "aaab"
- "aaba"
- "aabb"
- "abaa"
- "abab"
- "abba"
- "abbb"
- "bbaa"
- "bbab"
- "bbba"
- "bbbb"
- "aaaaa"
- "aaaab"
- "aaaba"
- "aaabb"
- "aabaa"
- "aabab"
- "aabba"
- "aabbb"
- "abaaa"
- "abaab"
- "ababa"
- "ababb"
- "abbaa"



MERRIMACK COLLEGE

Suggestion: Try by yourself all words up to size 8.

Models of Computation

n Mathematicians are a bit like the laconic Vermonter who, when asked if he's lived in the state his whole life, replies, 'Not yet.'

John Allen Paulos

Since you are here in this course, you have been working with programming and discrete mathematics for a while.

However, it is only now that you will be aware of what computation is about, *i.e.*, what indeed computers can do, from a mathematical and logical point of view.

- Theory of Computation
- Languages
- **Examples**



What can be described by FSA?



The FSA, as seen, are capable of locating sequences within a word, either to consider the word belonging to a language or not.

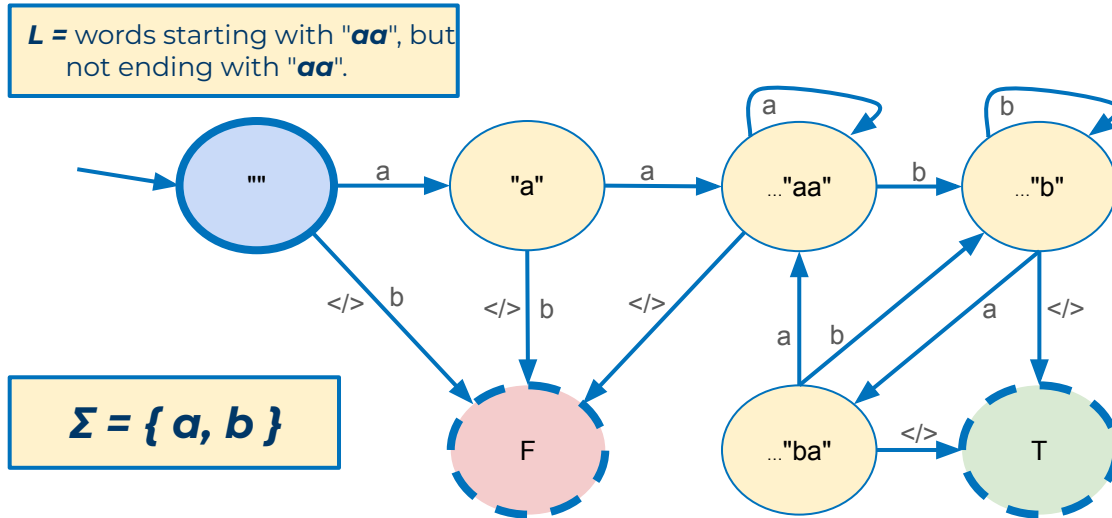
How would it be a FSA over the alphabet

$$\Sigma = \{a, b, c\}$$

for the language with no empty words nor double letters in it?

For example:

- **abcb** is valid;
- **bcaacb** is invalid.

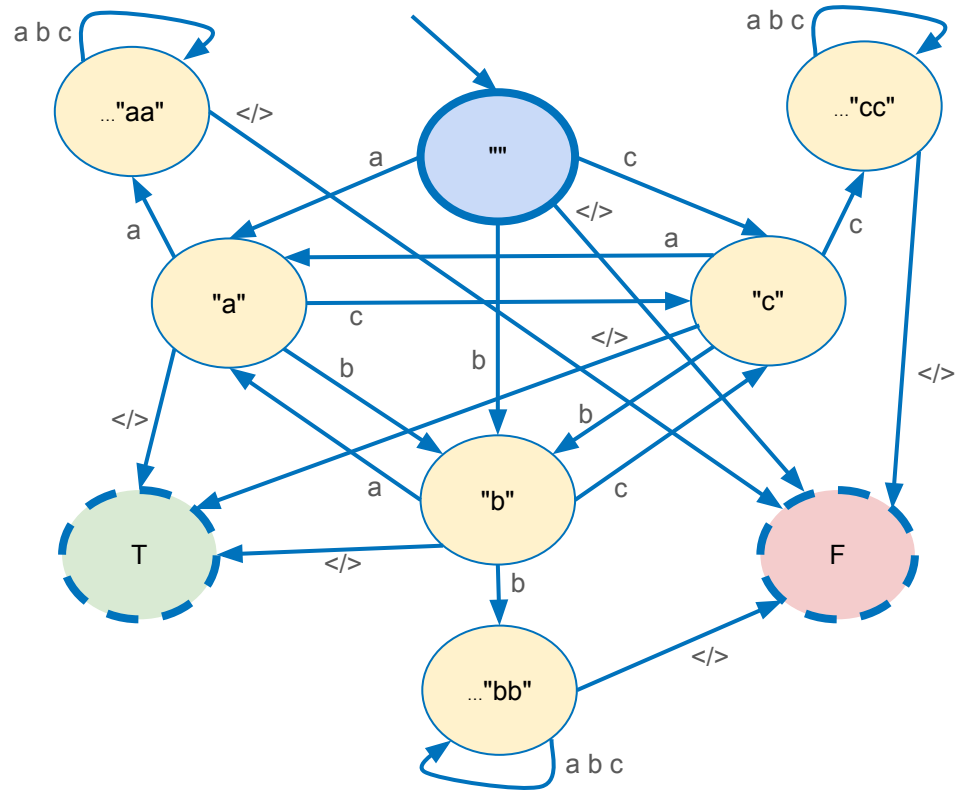


Solution

$$\Sigma = \{a, b, c\}$$

L = non empty words without double letters in it (no **"aa"**, **"bb"**, nor **"cc"**).

- **"** is invalid;
- **a** is valid;
- **abc** is valid;
- **aabcab** is invalid;
- **abcabb** is invalid;
- **abccab** is invalid;
- **abcab** is valid.



What can be described by FSA?

You saw in the recommended video (see slide 10) that you could develop a FSA to verify if a word is a valid variable identifier, which is a simple case:

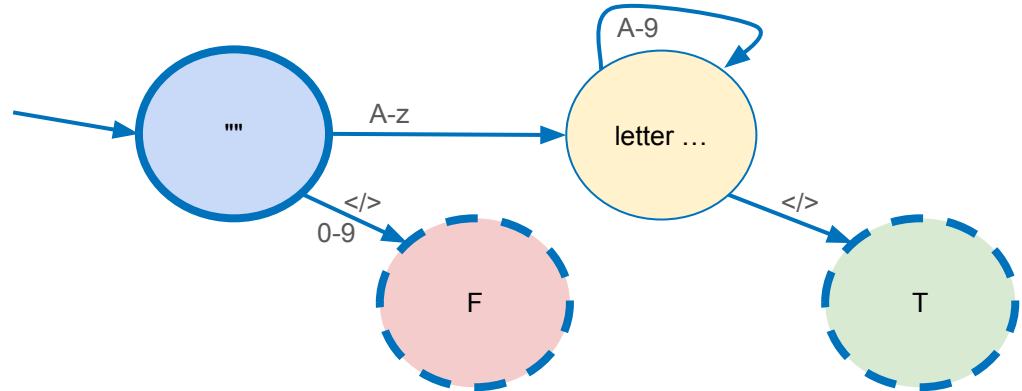
- $\Sigma = \{ A, B, C, \dots, Z, a, b, \dots, z \} \cup \{ 0, 1, 2, \dots, 8, 9 \};$
- $L =$ all words that start with a letter, followed by either a letter or a digit.



Solution

A FSA to verify if a word is a valid variable identifier, which is a simple case:

- $\Sigma = \{ A, B, C, \dots, Z, a, b, \dots, z \} \cup \{ 0, 1, 2, \dots, 8, 9 \};$
- $L =$ all words that start with a letter, followed by either a letter or a digit.



Finite Automata

“Mathematics is no more computation than typing is literature.”

John Allen Paulos

A Finite State Automaton (or machine) is a computational model. It allows recognize the words of a language based on sequence of characters.

Despite being powerful, Finite Automata cannot encompass everything a computer do. There are more powerful machines available. So, now let's explore the limits of FSA.

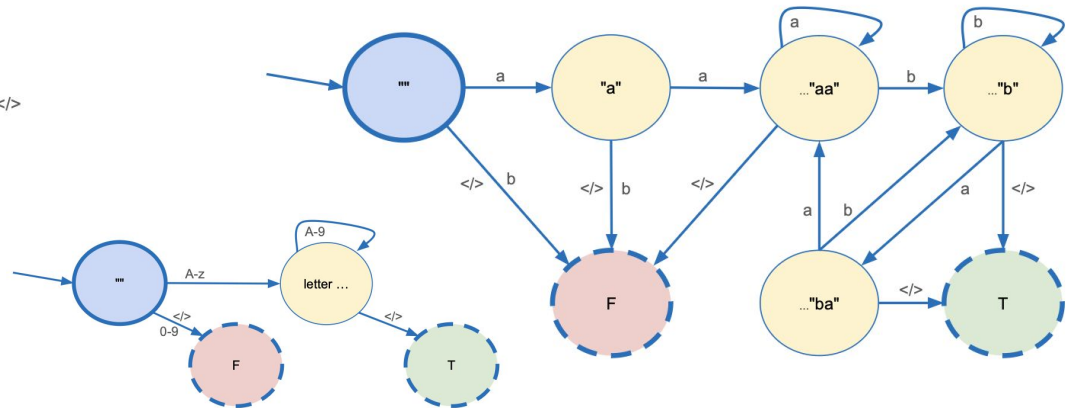
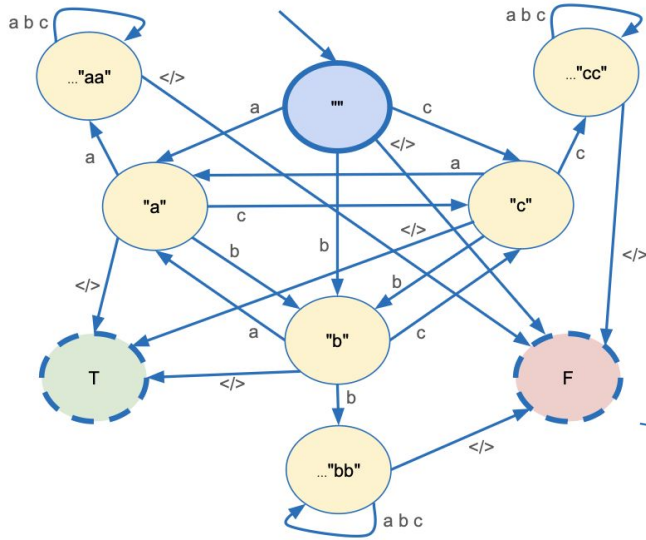
- **Deterministic and Non-Deterministic**
- Models of Computation Hierarchy
- Implementation



Deterministic Finite Automata - DFA

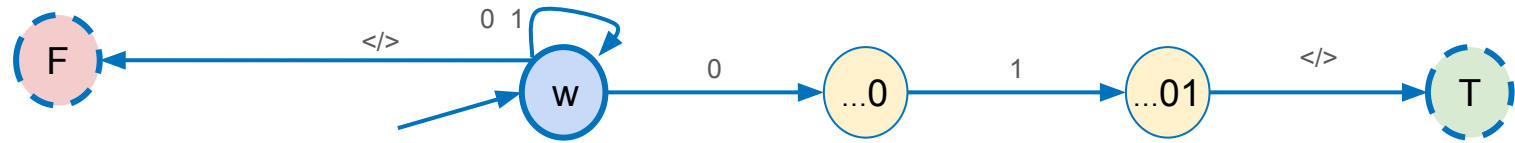
All FSA seem so far have to each state one only exiting arc to each possible alphabet element.

Thus, they are *Deterministic Finite Automata*.
We always know where to go next!



Non-Deterministic Finite Automata - NFA

In a *Non-Deterministic Finite Automata* there may be multiple exiting arcs with the same symbol. For example, considering the alphabet $\Sigma = \{0, 1\}$ the NFA below accept words ending with the sequence "**01**".



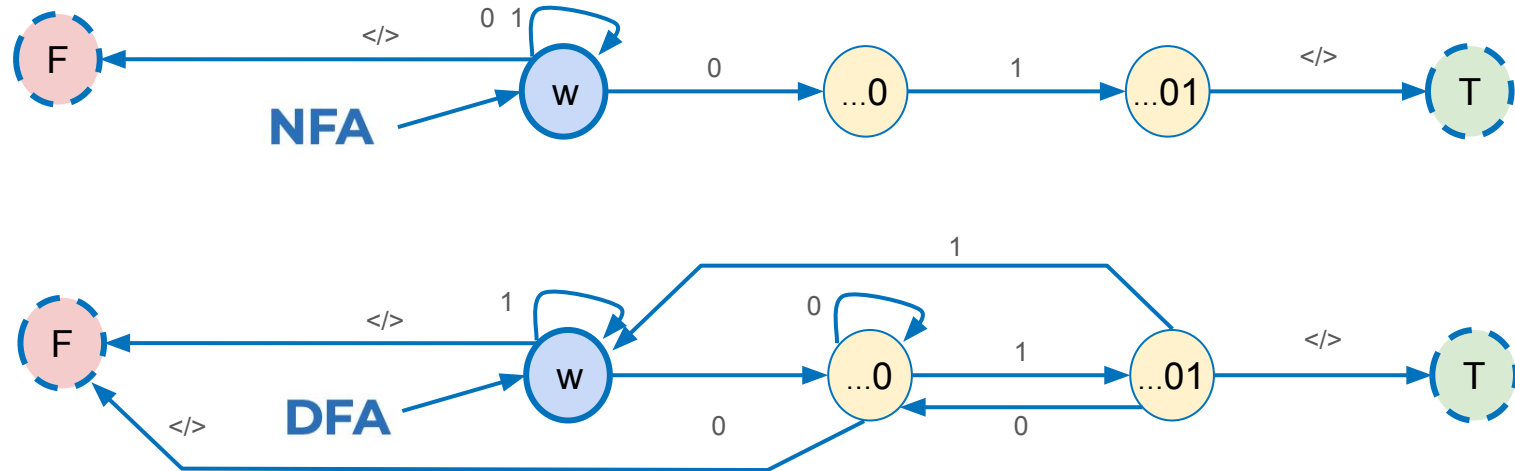
Note that in a NFA:

- It is possible to have more than one exiting arc associated to an alphabet letter, for example from the initial state **w** it is possible to stay in the state with letter **0**, or go to state **...0** with letter **0** too;
- It is possible to end the word and return **F** with the sequence "**01**", but there is a path to end the word with the sequence "**01**" and return **T**.



NFA and DFA are equivalent

It is proven that any DFA has at least one equivalent NFA, and, likewise, any NFA has at least one equivalent DFA.



Finite Automata

“Mathematics is no more computation than typing is literature.”

John Allen Paulos

A Finite State Automaton (or machine) is a computational model. It allows recognize the words of a language based on sequence of characters.

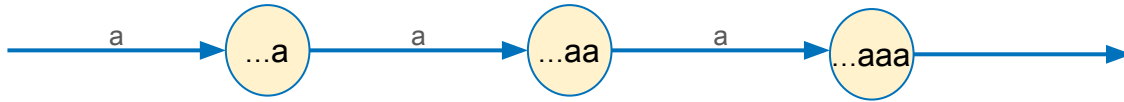
Despite being powerful, Finite Automata cannot encompass everything a computer do. There are more powerful machines available. So, now let's explore the limits of FSA.

- Deterministic and Non-Deterministic
- ***Models of Computation Hierarchy***
- Implementation



What a FSA cannot do?

FSA (DSA or NFA) can recognize patterns and each specific fixed pattern of size n letters requires a suite of n states. For example a pattern "**aaa**" requires three states.



However, since FSA are finite (finite number of states), only finite patterns could be recognized.



What a FSA cannot do?

How could a FSA recognize a language as:

- $\Sigma = \{0, 1, 2, 3, 4, <, >\}$
- L = all words with a letter $<$ will eventually have a letter $>$, so, a word with n letters $<$ needs to have n letters $>$.

How could a FSA recognize a language where all words are palindromes? For example: "civic", "level", and "madam".

How to count with FSA?

The simple answer is, you can only count with states that keep the memory of how many have been accounted already. Thus, you can only count a finite number.

For example:

- $<12>$
- $<1<34>5>$
- $2<03>5<>$
- $0<1<2<3>>>$
- $45<<>1><2>$



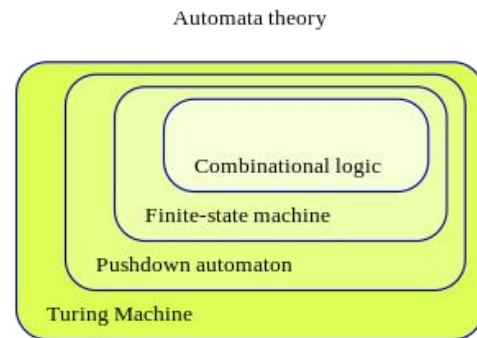
Models of Computation Hierarchy

Finite State Automata can recognize only languages that can be expressed as regular languages.

- Regular languages can be expressed by *Regular Expressions*;

To deal with more complex languages it is necessary to have a more powerful model of computation.

- Context-free grammars can be recognized through *Pushdown Automata*;
- Recursively enumerable languages can be recognized by a *Turing Machine*.



Finite Automata

“Mathematics is no more computation than typing is literature.”

John Allen Paulos

A Finite State Automaton (or machine) is a computational model. It allows recognize the words of a language based on sequence of characters.

Despite being powerful, Finite Automata cannot encompass everything a computer do. There are more powerful machines available. So, now let's explore the limits of FSA.

- Deterministic and Non-Deterministic
- Models of Computation Hierarchy
- ***Implementation***



Implementing a FSA

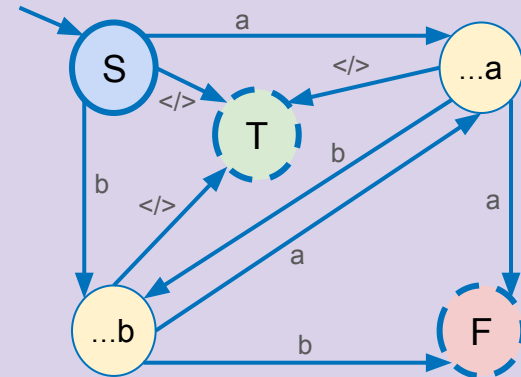
An implementation of a FSA can be done assuming a few format options:

- a stream input delivers a string with all letters of a word followed by an end symbol;
- the FSA is defined by:
 - a set of states;
 - an alphabet;
 - an initial state;
 - a subset of final states;
 - a transition function.
- a result stating if the input word belongs to the language accepted by the FSA.

We will start with a simple FSA that is defined by:

$$\Sigma = \{a, b\}$$

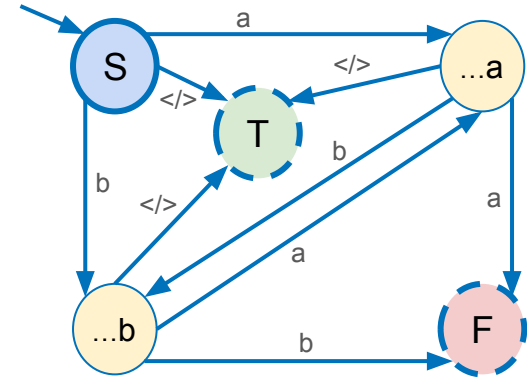
L = all words that have no double letter.



Implementing a FSA

- a stream input delivers a string with all letters of a word followed by an end symbol;

$\Sigma = \{a, b\}$ $\text{</>} = .$



- the FSA is defined by:
 - a set of states;
 - an alphabet;
 - an initial state;
 - a subset of final states;
 - a transition function.
- a result stating if the input word belongs to the language accepted by the FSA.

```
class FSA {
```

```
enum State { Start, dotA, dotB, retT, retF };
```

```
"Enter a string (alphabet {a, b}) followed by '.': "
```

```
State currentState = Start;
```

```
return (currentState == retT) || (currentState == retF);
```

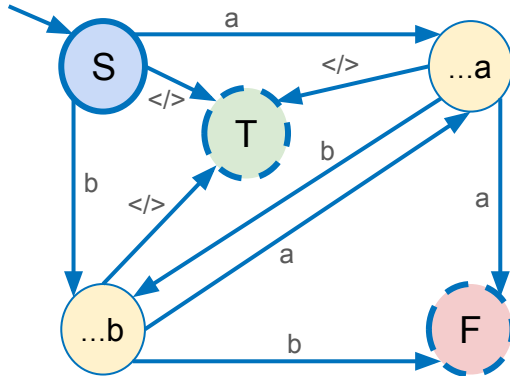
```
"String is accepted (no double letters)"
```

```
"String is not accepted (double letters)"
```



Implementing a FSA

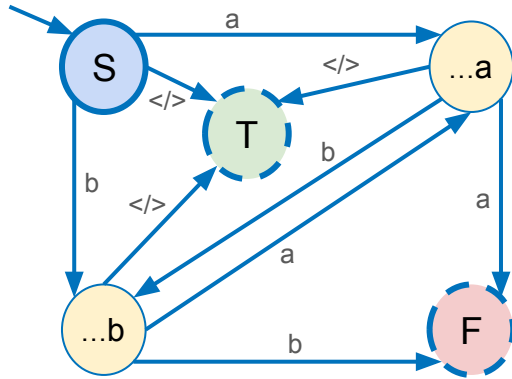
- a transition function.



```
10- bool ProcessChar(char c) {
11-     switch (currentState) {
12-     case Start:
13-         if (c == 'a') {
14-             currentState = dotA;
15-         } else if (c == 'b') {
16-             currentState = dotB;
17-         } else if (c == '.') {
18-             currentState = retT;
19-         } else {
20-             currentState = retF;
21-         }
22-         break;
23-     case dotA:
24-         if (c == 'a') {
25-             currentState = retF;
26-         } else if (c == 'b') {
27-             currentState = dotB;
28-         } else if (c == '.') {
29-             currentState = retT;
30-         } else {
31-             currentState = retF;
32-         }
33-         break;
34-     case dotB:
35-         if (c == 'a') {
36-             currentState = dotA;
37-         } else if (c == 'b') {
38-             currentState = retF;
39-         } else if (c == '.') {
40-             currentState = retT;
41-         } else {
42-             currentState = retF;
43-         }
44-         break;
45-     }
46-     return (currentState == retT) || (currentState == retF);
47- }
```



Implementing a FSA



```
1 #include <iostream>
2 #include <string>
3
4 class FSA {
5 private:
6     enum State { Start, dotA, dotB, retT, retF };
7 public:
8     State currentState = Start;
9
10    bool ProcessChar(char c) {==}
11
12    bool IsAccepted() const {
13        return currentState == retT;
14    }
15 };
16
17 int main() {
18     FSA fsa;
19     std::string input;
20
21     std::cout << "Enter a string (alphabet {a, b}) followed by '.' : ";
22     std::cin >> input;
23
24     for (char c : input)
25     {
26         if (fsa.ProcessChar(c)) {
27             if (fsa.IsAccepted())
28                 std::cout << "String is accepted (no double letters)" << std::endl;
29             else
30                 std::cout << "String is not accepted (double letters)" << std::endl;
31             break;
32         }
33     }
34     return 0;
35 }
```



This Week's tasks

- In-class Exercise E#3
- Coding Project P#3
- Quiz Q#3

Tasks

- Create a FSA to count coins.
- Create a C++ program to implement a FSA to recognize a language.
- Quiz #3 about this week topics.



In-class Exercise - E#3

Based on the example in slide 10, create a FSA that receives the more common US coins and delivers the acknowledgement that 30 cents were correctly entered in the first coins.

Your FSA can be drawn by hand or using any graphical editor of your choice, but either way it has to be understandable.

If I cannot read it, I cannot grade it with something other than zero!

Once the FSA is duly depicted, save the image into a **.pdf** file.

You have to submit the **.pdf** file with your FSA on Canvas within the deadline.

COIN	NAME	VALUE
	Penny	1¢
	Nickel	5¢
	Dime	10¢
	Quarter	25¢



MERRIMACK COLLEGE

This task counts towards the In-class Exercises grade and the deadline is This Saturday.

Third Coding Project - P#3

Following the example in the previous slides, create your own FSA that acknowledges the words of a language over the alphabet **{ a, b, c }** where all the words are at least three letters long and have a sequence with three letters **a** or three letters **b**.

After that, build a C++ program with the definition of a class **FSA** that implements your FSA. The class should have at least:

- The enum for the states,
- A method for processing the chars,
- A method for deciding if the word is accepted or rejected.

Create also a **main** function that receives a string to be analyzed and delivers the result computed by the FSA.

Your task: Go to Canvas, and submit your C++ file (.cpp) within the deadline.



MERRIMACK COLLEGE

This assignment counts towards the Projects grade and the deadline is Next Tuesday.

Third Quiz - Q#3

- The third quiz in this course covers the topics of Week 3;
- The quiz will be available this Saturday, and it is composed by 10 questions;
- The quiz should be taken on Canvas (Module 3), and it is not a timed quiz:
 - You can take as long as you want to answer it;
- The quiz is open book, open notes, and you can even use any language Interpreter to answer it;
- Yet, the quiz is evaluated and you are allowed to submit it only once.

Your task:

- Go to Canvas, answer the quiz and submit it within the deadline.



MERRIMACK COLLEGE

This quiz counts towards the Quizzes grade and the deadline is Next Tuesday.

” Week 3 of CSC 6033

- Do In-class Exercise E#3 until Saturday;
- Do Quiz Q#3 (available Saturday) until next Tuesday;
- Do Coding Project P#3 until next Tuesday.

Next Week - Regular Expressions



MERRIMACK COLLEGE